Correção · Algoritmos · Listas, pilhas e filas · 10 questões (diagnóstico)

Data: 11/11/2026 (bloco de 10/11) · **Semana 3** · **Placar: 8/10 (80%)** · 1 erro · 1 "não sabia" ("?")

Respostas lidas das anotações do PDF: 1-B · 2-D · 3-A · 4-E · 5-C · 6-"?" · 7-A · 8-E · 9-B · 10-A

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais.

> **Como li as marcas:** usei os círculos nas letras (a tabela "Minhas respostas" ficou vazia). **Q6:** só "?" e o desenho de um vetor `[ . . . . . ]`, sem círculo = **não sabia**. **Q10:** círculo na (A) com **pontinho** ao lado = candidata sem certeza. Os pontinhos da Q5 estão dentro das casas do vetor que você desenhou (rastreio), não ao lado de alternativa. As outras 8 foram marcadas **sem dúvida**, várias com o rastreio certo na margem (Q5, Q7, Q8, Q9).

Questão	Assunto	Sua	Gabarito	Resultado
1	Pilha em código (append/pop)	B	B	✅
2	Fila de prioridade (desempate por chegada)	D	D	✅
3	ArrayList × LinkedList (inserção no início)	A	A	✅
4	Parênteses balanceados com pilha (código)	E	E	✅
5	Fila circular em vetor (código, % N)	C	C	✅
6	Voltar/Avançar = duas pilhas	?	C	▫️ não sabia
7	Remoção em lista encadeada (código Java)	A	A	✅
8	Complexidade da lista simplesmente encadeada	E	E	✅
9	Fila + pilha em código (deque + lista)	B	B	✅
10	list.pop(0) × deque.popleft()	A	D	❌ (com dúvida)

Por tipo:

Tipo	Questões	Placar
Leitura de código (rastrear)	1, 4, 5, 7, 9	**5/5**
Escolha da estrutura pelo cenário	2, 6	1/2 (Q6 "?")
Complexidade das operações	3, 8, 10	2/3

Acertos firmes: 8 de 8 (nenhum acerto com pontinho).

Comparação: é o primeiro bloco do assunto (diagnóstico, antes da teoria). Na mesma disciplina, Big-O (06/11) foi **7/12 (58%)** → listas, pilhas e filas **8/10 (80%)**. A faculdade deixou base firme aqui: **leitura de código 5/5**, logo depois de você ter errado o `range` no simulado (Q51). O único erro (Q10) é o **mesmo princípio que você acertou na Q3**: mexer no início de um vetor desloca todos os elementos.

Teoria mínima do assunto (para fechar o item da fila)

> Como o diagnóstico saiu 8/10 com acertos firmes, esta seção substitui a aula: é o que a FGV cobra de listas, pilhas e filas.

Estrutura	Regra de saída	Uso típico no enunciado	Python	Java
Pilha	LIFO: sai o último que entrou	desfazer, voltar, parênteses, chamada de função (recursão)	`list.append` + `list.pop()`	`Deque.push/pop` (ou `Stack`)
Fila	FIFO: sai o primeiro que entrou	ordem de chegada, impressão, atendimento	`deque.append` + `deque.popleft()`	`Queue.offer/poll` (`ArrayDeque`, `LinkedList`)
Fila de prioridade	sai o de maior prioridade; empate pela chegada	idoso/PcD, urgência, agendador	`heapq`	`PriorityQueue`
Deque	entra e sai pelas duas pontas	janela deslizante, histórico com limite	`collections.deque`	`ArrayDeque`
Fila circular	FIFO em vetor fixo, índice volta ao 0	buffer de tamanho fixo	`(i + 1) % N`	`(i + 1) % N`

Operação	Vetor (list, ArrayList)	Lista encadeada (só head)	Lista encadeada (head + tail)
Acesso por índice	**O(1)**	O(n)	O(n)
Inserir/remover no **início**	**O(n)** (desloca todos)	**O(1)**	O(1)
Inserir no **fim**	O(1) amortizado	**O(n)** (percorre até o último)	**O(1)**
Remover do fim	O(1)	O(n)	O(n) na simples · O(1) na dupla
Buscar valor (não ordenada)	O(n)	O(n)	O(n)

📌 **Pegadinhas FGV:** (1) "lista encadeada tem acesso O(1) por índice" é falso · (2) lista encadeada **não ordenada** nunca tem busca O(log n) (busca binária exige vetor ordenado) · (3) `list.pop(0)`, `list.insert(0, x)` e `ArrayList.add(0, x)` são O(n) · (4) "último aponta para o primeiro" (lista circular) **não** define ordem de atendimento.

Questão 1 · ✅ Acertou

O sistema e-PROTOCOLO guarda, em uma pilha, os documentos abertos pelo servidor, para permitir a função "desfazer". O analista Rogério testou o trecho em Python a seguir, em que a lista é usada como pilha (append empilha e pop desempilha):

```
pilha = []
for doc in ["P1", "P2", "P3"]:
    pilha.append(doc)
pilha.pop()
pilha.append("P4")
pilha.append("P5")
x = pilha.pop()
y = pilha.pop()
print(x, y, pilha)
```

Ao final da execução, será impresso:

(A) P2 P3 ['P4', 'P5'];

✅ **(B) P5 P4 ['P1', 'P2'];** ← sua resposta

(C) P4 P5 ['P1', 'P2'];

(D) P5 P4 ['P1', 'P2', 'P3'];

(E) P1 P2 ['P3', 'P4', 'P5'].

Por quê: [P1, P2, P3] → pop tira P3 → [P1, P2] → empilha P4 e P5 → x = P5, y = P4 → sobra [P1, P2]. A (C) inverte a ordem de saída (seria fila), a (D) esquece o primeiro pop e a (A) e a (E) tiram de baixo.

📌 Pilha: o `pop()` sempre tira o topo (o último `append`). Dois pops seguidos saem na ordem **inversa** da entrada (P5 antes de P4).

Questão 2 · ✅ Acertou

O sistema e-BALCÃO, do balcão virtual de um Tribunal de Justiça, atende as solicitações por ordem de chegada. Por determinação legal, porém, os pedidos de pessoas idosas ou com deficiência devem ser atendidos antes dos demais; entre os pedidos de mesma prioridade, continua valendo a ordem de chegada. A analista Carla precisa escolher a estrutura de dados que organiza a fila de atendimento.

A estrutura mais adequada é:

(A) pilha, pois o último pedido prioritário a chegar deve ser o primeiro a ser atendido;

(B) fila simples (FIFO), pois a ordem de chegada é o único critério de atendimento;

(C) deque, pois os pedidos podem ser inseridos e removidos por qualquer das extremidades, sem critério de ordem;

✅ **(D) fila de prioridade, em que sai primeiro o pedido de maior prioridade e, entre pedidos de mesma prioridade, o que chegou antes;** ← sua resposta

(E) lista circular, pois o último pedido aponta para o primeiro, o que garante que todos sejam atendidos.

Por quê: dois critérios (prioridade legal + chegada) = fila de prioridade com desempate FIFO. A (B) ignora a prioridade, a (A) atenderia o último a chegar, a (C) não tem critério e a (E) descreve ligação entre nós, não ordem de atendimento. Você riscou as três erradas pela palavra certa ("pilha", "deque", "circular").

📌 "Prioridade + entre iguais, ordem de chegada" = **fila de prioridade** (heap). Só "ordem de chegada" = fila FIFO.

Questão 3 · ✅ Acertou

No sistema e-PAUTA, escrito em Java, a operação mais frequente sobre a lista de audiências do dia é a inclusão de uma audiência urgente no **início** da lista. O acesso a uma audiência pela posição (índice) é raro. O analista Vítor avalia substituir o ArrayList atual por um LinkedList.

Sobre essa decisão, é correto afirmar que o analista deve escolher:

✅ **(A) LinkedList, pois a inserção no início é O(1), enquanto no ArrayList é O(n), já que todos os elementos precisam ser deslocados;** ← sua resposta

(B) ArrayList, pois a inserção no início de um vetor dinâmico é O(1), por causa da capacidade reservada;

(C) LinkedList, pois o acesso por índice na lista encadeada é O(1), o que também acelera a inserção;

(D) ArrayList, pois a LinkedList exige O(n) para inserir no início, já que precisa percorrer a lista até o último nó;

(E) qualquer das duas, pois ambas têm inserção no início em O(1) e acesso por índice em O(1).

Por quê: você sublinhou as duas pistas do cenário: "inclusão no **início**" (frequente) e "índice é raro". Encadeada insere no início só trocando o ponteiro head: O(1). No ArrayList, todo mundo anda uma casa: O(n). A (B) confunde capacidade reservada (ajuda no **fim**), a (C) e a (E) dão O(1) por índice à encadeada e a (D) troca início por fim.

📌 Vetor (ArrayList, list) = índice O(1), mas **início O(n)**. Encadeada = início O(1), mas índice O(n). O cenário diz qual operação é frequente.

Questão 4 · ✅ Acertou

O analista Gustavo escreveu a função a seguir para o validador de fórmulas do sistema e-CÁLCULO, que verifica se parênteses, colchetes e chaves de uma expressão estão balanceados:

```
def valida(expr):
    pares = {')': '(', ']': '[', '}': '{'}
    pilha = []
    for c in expr:
        if c in '([{':
            pilha.append(c)
        elif c in ')]}':
            if not pilha or pilha.pop() != pares[c]:
                return False
    return len(pilha) == 0
```

A chamada que retorna True é:

(A) valida("([)]");

(B) valida("{[()]}");

(C) valida("(()");

(D) valida("][()");

✅ **(E) valida("{[()()]}").** ← sua resposta

Por quê: (A) e (B) fecham na ordem errada (o topo é `[` ou `(` e chega o outro fechamento) · (C) sobra um `(` na pilha · (D) começa com `]` e a pilha está vazia. Na (E), cada fechamento encontra o seu par no topo e a pilha termina vazia. Você anotou "HASH" ao lado do `pares`: está certo, o `dict` é uma tabela hash (consulta O(1) médio), e isso já liga com o bloco 03.

📌 Balanceamento = **pilha**: abre empilha, fecha desempilha e compara. Três jeitos de dar False: fechamento com pilha vazia, par trocado no topo e pilha que sobra no fim.

Questão 5 · ✅ Acertou

O sistema e-INTIMA usa uma fila circular, implementada em um vetor de 5 posições, para enfileirar as intimações a serem enviadas. Considere o código Python a seguir:

```
N = 5
fila = [None] * N
inicio, fim, qtd = 0, 0, 0

def enfileira(x):
    global fim, qtd
    fila[fim] = x
    fim = (fim + 1) % N
    qtd += 1

def desenfileira():
    global inicio, qtd
    x = fila[inicio]
    inicio = (inicio + 1) % N
    qtd -= 1
    return x
```

A analista Renata executou, nesta ordem: enfileira(10), enfileira(20), enfileira(30), enfileira(40), desenfileira(), desenfileira(), enfileira(50), enfileira(60) e enfileira(70).

Após essa sequência, os valores de inicio e de fim e o próximo valor a ser desenfileirado são, respectivamente:

(A) 2, 7 e 30;

(B) 0, 2 e 10;

✅ **(C) 2, 2 e 30;** ← sua resposta

(D) 2, 2 e 60;

(E) 4, 2 e 50.

Por quê: 10, 20, 30, 40 nas casas 0–3 (fim = 4) · dois desenfileira → inicio = 2 · 50 na casa 4, fim = (4 + 1) % 5 = **0** · 60 na casa 0, fim = 1 · 70 na casa 1, fim = 2. O próximo a sair está em fila[2] = 30. Você desenhou as 5 casas com I e F: é o método. A (A) esquece o `% N` (fim = 7), a (D) acha que sai o último inserido na casa 0 e a (B) e a (E) erram o inicio.

📌 Fila circular: o índice anda com `(i + 1) % N` e volta ao 0. O próximo a sair é sempre `fila[inicio]`, mesmo que o fim tenha dado a volta. Desenhar as N casas e mover I e F.

Questão 6 · ⬜ Não sabia ("?")

O visualizador de autos do sistema JUSTIÇAPB+ tem os botões "Voltar" e "Avançar" para navegar entre as peças já abertas pelo magistrado, como em um navegador web. Quando uma nova peça é aberta, o histórico de "Avançar" é descartado. O analista Sérgio precisa implementar o recurso com inserção e remoção em tempo O(1).

A solução mais adequada é usar:

(A) uma única fila, em que "Voltar" remove do início e "Avançar" insere no fim;

(B) uma fila de prioridade, ordenada pela data de juntada de cada peça;

✅ **(C) duas pilhas, uma para "Voltar" e outra para "Avançar", esvaziando a pilha de "Avançar" quando uma nova peça é aberta;**

(D) duas filas, uma para "Voltar" e outra para "Avançar", esvaziando a fila de "Voltar" quando uma nova peça é aberta;

(E) uma tabela hash, indexada pelo número da peça, que guarda a ordem de visita de cada peça.

Onde travou: você desenhou um vetor `[ . . . . . ]` com as peças em sequência e pôs o "?". A ideia de uma lista com um cursor andando para os lados funciona, mas não era o que as alternativas ofereciam, e faltou a pergunta-chave: **"Voltar" leva para qual peça?** Para a **mais recente** antes da atual. "Mais recente primeiro" = **LIFO = pilha**.

Como funciona com duas pilhas (peça atual fora das pilhas):

Ação	Pilha Voltar	Atual	Pilha Avançar
Abre P1, P2, P3	[P1, P2]	P3	[ ]
Voltar	[P1]	P2	[P3]
Voltar	[ ]	P1	[P3, P2]
Avançar	[P1]	P2	[P3]
Abre P4 (nova)	[P1, P2]	P4	[ ] ← esvazia

Eliminação que dava: (B) e (E) não guardam a ordem de navegação; (A) e (D) usam fila, que devolveria a peça **mais antiga** (FIFO). A (D) ainda esvazia a pilha errada: o enunciado diz que o descartado é o "Avançar".

📌 **Voltar/Avançar (navegador) e Desfazer/Refazer = DUAS PILHAS.** Abrir nova: empilha a atual em Voltar e **esvazia Avançar**. Voltar: empilha a atual em Avançar e desempilha de Voltar. Avançar: o inverso. Push e pop são O(1).

Questão 7 · ✅ Acertou

A analista Beatriz revisa o método a seguir, do sistema SISGUIAS, escrito em Java, que remove o nó seguinte a um nó informado de uma lista simplesmente encadeada:

```
class No {
    int valor;
    No prox;
}

void removeApos(No ant) {
    if (ant != null && ant.prox != null) {
        ant.prox = ant.prox.prox;
    }
}
```

A lista contém, nesta ordem, os valores 3 → 8 → 5 → 9 → null, e a variável p aponta para o nó de valor 3. Em seguida, são executadas as chamadas removeApos(p.prox) e removeApos(p), nessa ordem.

Ao final, a lista será:

✅ **(A) 3 → 9 → null;** ← sua resposta

(B) 3 → 5 → 9 → null;

(C) 3 → 8 → 9 → null;

(D) 8 → 9 → null;

(E) 9 → null.

Por quê: p.prox é o nó 8, então a 1ª chamada remove o que vem **depois do 8** (o 5) → 3 → 8 → 9. A 2ª remove o que vem depois do 3 (o 8) → 3 → 9. Você riscou o 5 na lista do enunciado: rastreio certo. A (C) para na 1ª chamada, a (B) remove o 8 primeiro e a (D)/(E) removem o próprio 3, que o método nunca toca.

📌 `ant.prox = ant.prox.prox` "pula" o nó seguinte: remove **o vizinho da frente**, nunca o próprio `ant`. Por isso a remoção em lista encadeada é O(1) quando já se tem o nó anterior.

Questão 8 · ✅ Acertou

O sistema e-ARQUIVO mantém os processos arquivados em uma lista simplesmente encadeada com n nós. A implementação guarda apenas uma referência para o primeiro nó (head) e não guarda referência para o último nó. A lista não é mantida ordenada.

As complexidades de tempo das operações de inserir um processo no início da lista, inserir um processo no fim da lista e buscar um processo pelo número são, respectivamente:

(A) O(1), O(1) e O(1);

(B) O(n), O(1) e O(log n);

(C) O(1), O(1) e O(n);

(D) O(1), O(n) e O(log n);

✅ **(E) O(1), O(n) e O(n).** ← sua resposta

Por quê: você escreveu O(1), O(n) e O(n) em cima de cada operação antes de olhar as alternativas: método perfeito. Sem tail, inserir no fim exige percorrer até o último (O(n)), o que elimina (A) e (C). Lista **não ordenada** e sem índice não admite busca binária, o que elimina (B) e (D).

📌 Encadeada só com head: início O(1) · fim O(n) (com tail, O(1)) · busca O(n) sempre. O(log n) exige vetor **ordenado** com acesso por índice.

Questão 9 · ✅ Acertou

O analista Márcio revisa o trecho a seguir, em Python, do sistema e-REMESSA, que reorganiza a fila de lotes a serem enviados ao tribunal superior. Na lista, append empilha e pop desempilha:

```
from collections import deque

fila = deque([4, 7, 1, 9])
pilha = []
while fila:
    pilha.append(fila.popleft())
pilha.pop()
while pilha:
    fila.append(pilha.pop())
fila.append(fila.popleft())
print(list(fila))
```

Ao final da execução, será impresso:

(A) [1, 7, 4];

✅ **(B) [7, 4, 1];** ← sua resposta

(C) [1, 7, 9];

(D) [7, 1, 4];

(E) [1, 7, 4, 9].

Por quê: a pilha fica [4, 7, 1, 9] (topo 9) · `pop()` descarta o 9 · esvaziar a pilha na fila inverte → [1, 7, 4] · a última linha tira o 1 da frente e põe no fim → [7, 4, 1]. Você riscou o 9 e escreveu 1, 7, 4 na margem. A (A) é a armadilha de quem para antes da última linha; a (E) esquece o `pop()` do 9.

📌 Passar uma fila inteira por uma pilha **inverte a ordem**. `fila.append(fila.popleft())` é uma rotação: o primeiro vai para o fim.

Questão 10 · ❌ Errou (com dúvida)

O sistema e-DISTRIBUI processa, a cada madrugada, cerca de um milhão de processos em ordem de chegada. A analista Juliana comparou as duas versões a seguir, em Python:

```
# Versão 1
fila = []
for p in novos:
    fila.append(p)
while fila:
    distribui(fila.pop(0))

# Versão 2
from collections import deque
fila = deque()
for p in novos:
    fila.append(p)
while fila:
    distribui(fila.popleft())
```

A versão 1 mostrou-se muito mais lenta que a versão 2. A explicação correta é que:

(A) list.pop(0) é O(1), mas append em lista é O(n), o que torna lenta a montagem da fila na versão 1; ← sua resposta

(B) as duas versões têm a mesma complexidade assintótica, e a diferença se deve apenas a constantes de implementação;

(C) deque é implementado como pilha, e por isso popleft() remove o último elemento inserido em O(1);

✅ **(D) list.pop(0) é O(n), pois desloca todos os elementos restantes, enquanto deque.popleft() é O(1);**

(E) deque mantém os elementos ordenados, o que permite remover o menor elemento em O(log n).

Onde você se confundiu: você ligou "ordem de chegada" a "Fila" (certo) e marcou a (A) com pontinho, sem eliminar a (D). A (A) **inverte** as duas complexidades: a lista do Python é um **vetor** (como o ArrayList), então `append` no fim é O(1) e `pop(0)` tira do **início**, o que obriga os n − 1 restantes a andarem uma casa: O(n). Com um milhão de processos, isso vira ≈ $n^2/2$ deslocamentos. É exatamente o princípio que você acertou na **Q3** ("no ArrayList é O(n), já que todos os elementos precisam ser deslocados"). O `deque` é uma lista duplamente encadeada de blocos: tirar da frente só move o ponteiro, O(1).

Pista que levava à certa: as duas versões fazem os mesmos `append`. Se a montagem fosse o problema, a versão 2 também seria lenta. A única linha diferente é `pop(0)` × `popleft()`, então a diferença **tem** de estar na remoção. Isso elimina (A) e (B); (C) e (E) descrevem estruturas que não são deque (pilha e fila de prioridade).

📌 **Lista do Python = vetor:** `append` e `pop()` no **fim** = O(1) · `pop(0)` e `insert(0, x)` no **início** = O(n) (desloca todos). **Fila em Python = `collections.deque`** (`append` + `popleft`, ambos O(1)). Laço de n `pop(0)` = $O(n^2)$.

Para o caderno de erros

Assunto	Por que errei	Regra certa
list.pop(0) × deque (Q10)	Inverti as complexidades: achei que o append era o caro e o pop(0) barato	Lista = vetor: fim O(1), início O(n). Fila em Python = deque (popleft O(1))
Voltar/Avançar (Q6, "?")	Pensei num vetor com cursor; não perguntei "volta para qual?"	Volta para o mais recente = LIFO → duas pilhas; nova peça esvazia o Avançar

---

O que estudar agora

Quando	O que fazer
Agora, 10 min	Ler a **teoria mínima** acima (as duas tabelas) e refazer **Q6 e Q10 sem olhar**: na Q6, montar a tabela das duas pilhas com P1–P4; na Q10, escrever "fim O(1) / início O(n)" ao lado de cada linha
Almoço · Português	🔴 **G. Semântica e reescrita** (1º da fila de Português)
19h · Bloco 1	🔴 **APF funções de dados:** responder o bloco de 06/11 (8 questões) e mandar para correção
20h · Bloco 2	🔴 Diagnóstico de **árvores** (`2026-11-10_Semana-03_Bloco-02_Arvores`, 10 questões). Se sobrar tempo, o de **hash** (bloco 03, 8 questões) fica para amanhã
22:40 · revisão	Regras 📌 de hoje + revisão espaçada de 04/11 (testes e Venn) + 15 min de Anki
Dom 15/11 · 13h	Simulado de domingo (rotina do plano)